

EEET2588/EEET2687

Real-Time Systems Engineering

Lecture 2: Concurrent Processes - Threads and Message Passing

Dr Linh Tran - School of Science, Engineering & Technology

General Guide

- **Read the definition boxes first.** They explain terms that the original slides often assume.
- **Use the code skeletons as patterns.** They are not full solutions.
- **Focus on flow:** create threads, control scheduling, protect shared data, then communicate across processes and nodes.
- **Skip optional performance topics for now** unless your lab or project explicitly needs them.

Key idea

Real-time programming is not just C code. You must reason about when threads run, when they block, what data is shared and how processes exchange messages.

Terms used throughout

RTOS = Real-Time Operating System.

IPC = Interprocess Communication.

POSIX = Portable Operating System Interface, a standard API family used by Unix-like systems.

The learning path in one picture



Reading rule for concurrent code

Never assume which thread is faster or exactly when a context switch occurs. A correct design must work for all valid interleavings.

Process vs Thread

- **Process**: a program in execution, with its own code, heap, stack(s), open files and OS resources.

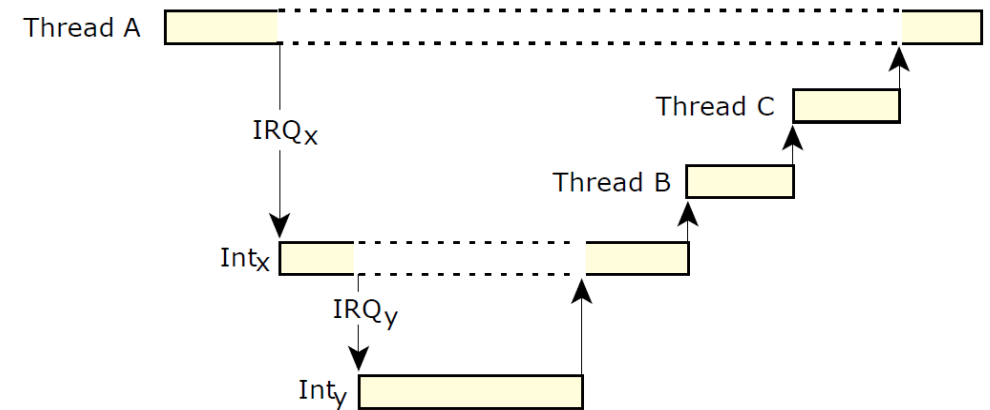
Separate processes improve modularity and memory protection because their address spaces are isolated.

- **Thread**: a flow of execution inside a process. Each process has at least one thread, usually the main thread.

Threads in one process share code, heap and global variables, but each thread has its own stack and CPU register context.

Common misunderstanding

A process is not “just a thread”. More precisely: a process owns resources, and one or more threads execute code within those resources.



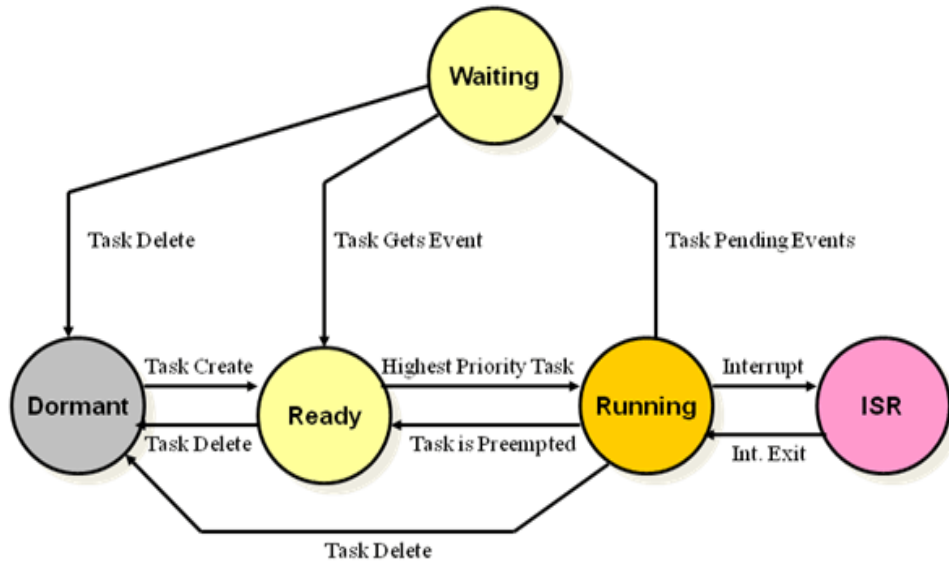
Example of thread execution timeline:

Thread A being interrupted by IRQx, followed by Intx being preempted by the higher-priority IRQy. When Intx finishes, the scheduler selects Thread B, followed by Thread C. After the other ready threads finish or block, Thread A resumes from where it was interrupted.

QNX IDs

getpid() returns the process ID.
gettid() returns the current thread ID.
getppid() returns the parent process ID.
gethostname() returns the node name.

Process and thread states



Generic RTOS state-transition diagram

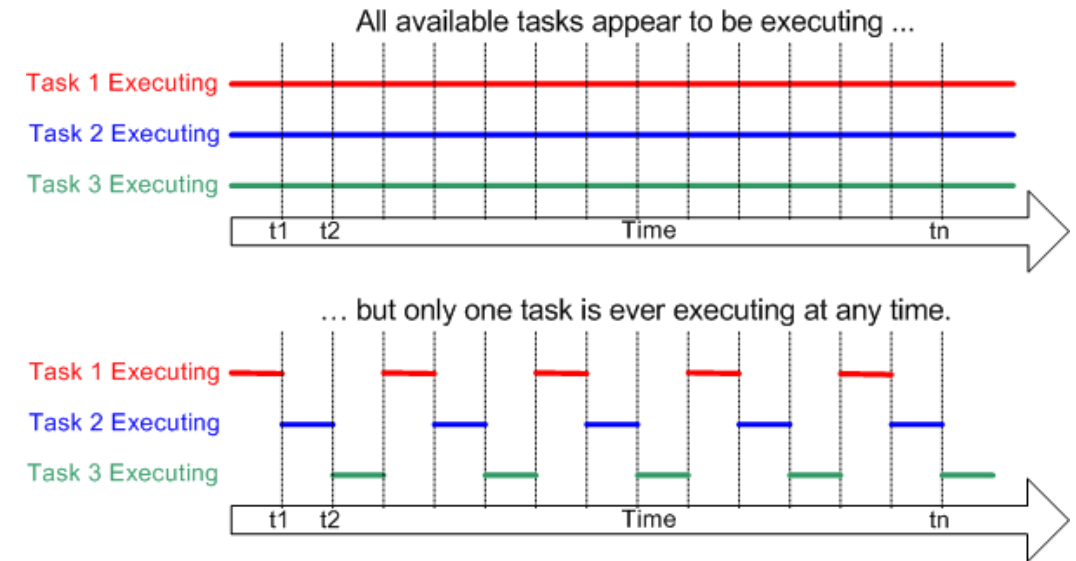
Simplified category	Meaning in practice
Running	Currently executing on a CPU. Only running threads consume CPU time.
Ready	Able to run, but waiting for the scheduler to assign a CPU.
Blocked	Waiting for an event, so it cannot run yet. QNX examples: NANOSLEEP, MUTEX, SEM, CONDVAR, SEND, RECEIVE or REPLY.
Stopped/Dead	Suspended, terminated, or not active. Resources may remain until cleaned up.

Important note

- This is a simplified RTOS state-transition diagram. It shows the main idea: a thread moves between **Ready**, **Running**, and **Blocked/Waiting** depending on scheduling and events. In QNX, there are about 21 detailed thread states in `<sys/neutrino.h>`
- **ISR** (Interrupt Service Routine) is not a normal thread state.
- A **blocked** thread does not use CPU time (only running threads do), but it can delay other work if another thread is waiting for it.

Concurrency and Parallelism

- **Quasi-concurrent (pseudo-concurrent)**
 - Single Core CPU (serial processing with time slicing or cooperative scheduling)
- **Truly Concurrent**
 - Multiple CPUs or Cores including distributed computing, parallel processing
- **Two major categories**
 - **Asynchronous concurrency:** tasks run independently and synchronize occasionally, typical of RT and distributed systems. Example: webserver
 - **Synchronous concurrency:** tasks work in lock-step, typical of parallel computation. Example: GPU based computation



Example of quasi-concurrent execution on one CPU: all tasks appear to progress, but only one task executes at any exact instant

Analysis rule

Never assume which thread is faster or when a context switch occurs. Sleep() changes timing, but it is not synchronization.
Correct concurrent code must work for all valid execution orders.

Scheduling, priority and preemption

- **Scheduler**: the OS component that chooses which Ready thread runs next.
- **Priority**: in QNX, 1 is low and 63 is high. Higher-priority RT threads should respond sooner.
- **Preemption**: a running thread is interrupted when a higher-priority thread becomes Ready.
- **Context switch**: the kernel saves one thread's CPU state and loads another's.

Real-time rule

The highest-priority **Ready** thread should be running. Incorrect priorities can cause missed deadlines or priority inversion.

Context switch trigger

Occurs after an interrupt, unblocking event, yield, sleep, message arrival or priority change.

POSIX scheduling policies supported by QNX

- **SCHED_FIFO**: A fixed-priority policy. A thread runs until it blocks, yields, exits, or is preempted by a higher-priority thread.
- **SCHED_RR**: Similar to FIFO, but equal-priority threads share CPU time using round-robin time slices.
- **SCHED_OTHER**: Supported by QNX and currently behaves like SCHED_RR, but is not recommended because its behavior may change.

POSIX threads: creating a thread

POSIX threads, usually called **pthread**s, are the standard C API for creating and controlling threads. QNX supports this API. To run a function as a new thread, pass it to **pthread_create()** instead of calling it directly.

pthread_create() creates a new thread and starts running a function in that thread.

Syntax: **pthread_create(thread_id, attributes, function_to_run, argument_to_pass);**

```
#include <pthread.h>
#include <stdio.h>
#include <unistd.h> //to use sleep()

void *worker(void *arg)
{
    int seconds = *((int *)arg);
    printf("Thread started\n");
    sleep(seconds);
    printf("Thread finished\n");
    return NULL;
}

int main(void)
{
    pthread_t tid;
    int delay = 2;
    pthread_create(&tid, NULL, worker, &delay);
    pthread_join(tid, NULL);
}
```

Thread routine

A pthread routine must have the form `void *routine(void *arg)`. It accepts one `void*` argument and returns a `void*` value.

void* and type casting

`void*` means “pointer to data of unknown type”. Cast it back to the expected type inside the thread, such as `(int *) arg` or `(my_struct *) arg`.

Key safety rule

pthread_create() passes only one argument pointer to the new thread. Use a struct when you need to pass multiple values.

The data pointed to by `arg` must remain valid until the thread has finished using it.

Passing data to a thread

pthread_create() accepts **only one argument pointer**. We can place related values in a struct and pass the address of that struct to the thread.

```
#include <pthread.h>
#include <stdio.h>
#include <stdlib.h>

typedef struct {
    int var1; int var2; int var3;
} data_bundle;

void *thread_ex(void *data){
    data_bundle *a = (data_bundle *)data;
    a->var1 = 12; a->var2 = 22; a->var3 = 32;
    printf("Thread finished\n");
    return NULL;
}

int main(void){
    pthread_t th;
    data_bundle data = {5, 6, 210};
    pthread_create(&th, NULL, thread_ex, &data);
    pthread_join(th, NULL);

    printf("var1 = %d\n", data.var1);
    printf("var2 = %d\n", data.var2);
    printf("var3 = %d\n", data.var3);
    printf("Main terminated\n");
    return EXIT_SUCCESS;
}
```

Why a struct pointer?

The API gives you one void* argument. **A struct packages several values into one address.**

Visibility after join

Because the thread receives the address, changes to struct fields are visible to the caller after pthread_join().

Safety warning

Do not pass a pointer to data that may go out of scope before the thread finishes.

If multiple threads access or update the same data at the same time, protect it with a mutex (will study in Lab 3).

pthread_join() **waits for a thread to finish.** Without it, main() may finish too early, and the whole process may terminate before the worker thread completes.

Syntax:

```
int pthread_join(pthread_t thread, void **return_val);
```

pthread_join(): waiting for a thread

pthread_join(tid, &retval) blocks the calling thread until tid terminates.

- Use pthread_join() so main() waits instead of terminating early and ending the process.
- The second argument collects the value returned by the thread or passed to pthread_exit().
- If a joinable thread is never joined, resources may remain allocated until the process exits.

In QNX real-time code, **pthread_timedjoin()** can fail when the join misses a deadline.

```
#include <pthread.h>
#include <stdio.h>
#include <stdlib.h>

void *worker(void *arg){
    int *result = malloc(sizeof(int));
    *result = 152;
    return result;
}

int main(void){
    pthread_t tid;
    void *retval = NULL;
    pthread_create(&tid, NULL, worker, NULL);
    pthread_join(tid, &retval);

    printf("Result = %d\n", *(int *)retval);
    free(retval);
    return EXIT_SUCCESS;
}
```

Blocked state and deadline check

When main() calls pthread_join(), the main thread becomes **Blocked** until the target thread finishes.

In QNX, pthread_timedjoin() is useful when waiting must have a time limit.

Joined vs Detached threads

```
#include <pthread.h>
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

void *worker(void *arg){
    sleep(5);
    printf("Worker finished\n");
    return NULL;
}

int main(void){
    pthread_t tid;
    pthread_attr_t attr;

    pthread_attr_init(&attr);
    pthread_attr_setdetachstate(&attr,
        PTHREAD_CREATE_DETACHED);

    /* Start worker, but will not wait for it
       and not collect its return value */
    pthread_create(&tid, &attr, worker, NULL);
    pthread_attr_destroy(&attr);

    printf("Main stays alive for 6 seconds\n");
    sleep(6);
    printf("Main finished\n");
    return EXIT_SUCCESS;
}
```

Thread type	What happens	When to use
Joinable	Another thread can wait using pthread_join() . After it finishes, it keeps some resources (such as its thread status and return value), until another thread joins it	Need result or controlled shutdown.
Detached	Cannot be joined. Its resources are released automatically when the thread terminates. However, if the process ends first, the detached thread is terminated with the process.	Fire-and-forget work with no return value.

- **Default in QNX examples:** *threads are joinable.*
- Use detached threads only when the process will stay alive long enough for the thread to finish.
- **Detached does not mean independent process.** It still has the **same process ID and shared address space.**

Thread attributes and scheduling policy

Thread attributes are settings prepared before `pthread_create()`. They can be used to set the new thread's **scheduling policy** and **priority**.

- **Priority** decides which Ready thread should run first.
- **Scheduling policy** decides how Ready threads at the same priority share the CPU.

In the QNX lab, priority is commonly set in the range **1 low** to **63 high**, with default around **10**

```
pthread_t tid;
pthread_attr_t attr;
struct sched_param param;

pthread_attr_init(&attr);
pthread_attr_setschedpolicy(&attr, SCHED_RR);

param.sched_priority = 15;
pthread_attr_setschedparam(&attr, &param);
pthread_attr_setinheritsched(&attr,
    PTHREAD_EXPLICIT_SCHED);

pthread_create(&tid, &attr, worker, NULL);
```

`pthread_attr_t`

A structure that stores **creation settings** such as *detach state, stack size, scheduling policy, and whether the new thread inherits scheduling from the parent thread (the thread that calls `pthread_create`)*

`struct sched_param`

A structure that stores **scheduling parameters**. One of the key field is **sched_priority**.

`PTHREAD_EXPLICIT_SCHED`

Forces the new thread to use the policy and priority stored in attr. *Without it, the thread may inherit the parent's scheduling.*

Runtime information and error checking

Function or value	Meaning
gethostname (name, len)	Returns the node or host name. Useful when using qnet or /net paths.
getpid ()	Returns the process ID. All threads in the same process have the same PID.
gettid ()	Returns the thread ID in QNX. Useful for debugging output.
errno	Global error code set by many POSIX/QNX functions when they return -1.
strerror (errno)	Converts an error number into a readable message.
EOK	QNX success value, usually 0, used by functions and replies.

```
int rc = pthread_create(&tid, NULL, worker, NULL);
if (rc != EOK) {
    printf("pthread_create failed: %s\n",
          strerror(rc));
}

mqd = mq_open("/test_queue", O_RDONLY);
if (qd == (mqd_t)-1) {
    printf("mq_open failed: %s\n",
          strerror(errno));
}
```

Two error styles

- Many pthread_* functions return the error code directly, so use **strerror(rc)**.
- Many POSIX/QNX functions return -1 and set errno, so use **strerror(errno)**.

Shared data and race conditions

Threads can communicate efficiently because they share memory.
That is also why shared data can be dangerous

- **Race condition:** the final result depends on the unpredictable interleaving of threads.
- **Critical section:** code that accesses shared data and must not be executed by two threads at the same time.
- **Shared variable:** data reachable by multiple threads, often a global variable or heap object.

EXAMPLE:

```
// Not safe if two threads run this
counter++;

// Why? It is really three steps:
tmp = counter;
tmp = tmp + 1;
counter = tmp;
```

```
pthread_mutex_t lock = PTHREAD_MUTEX_INITIALIZER;
```

```
pthread_mutex_lock(&lock);
counter++;
pthread_mutex_unlock(&lock);
```

Mutex

A mutual exclusion lock. It allows only one thread at a time into the protected critical section.

Synchronization primitive cheat sheet

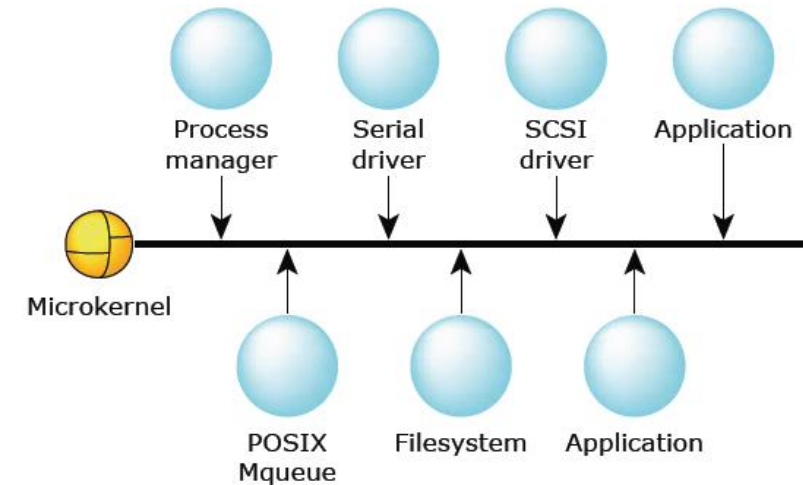
Primitive	Plain meaning	Typical use
Mutex	Allows only one thread at a time to enter a critical section	Protect shared variables such as counter, head and tail.
Reader/writer lock	Allows many readers at the same time, but only one writer.	Allow concurrent reads but exclusive writes.
Semaphore	A counter plus wait/post operations.	Count available resources or signal between threads.
Condition variable	A thread waits until a condition becomes true.	Producer-consumer not_empty and not_full.
Barrier	Makes N threads wait until all have reached the same point.	Wait until all worker threads reach the same point.

Important note

Condition variables must be checked in a **while** loop, not a single **if**, because another thread may change the condition before the woken thread runs.

IPC in QNX

- **IPC** means **Interprocess Communication**: mechanisms that let processes or threads exchange data and **synchronize execution**.
- **QNX is a microkernel RTOS**, so message passing is central to how system services and applications communicate.
- This lecture introduces two IPC styles: **POSIX message queues** and **native QNX message passing**.
- A design choice depends on blocking behavior, queueing needs, latency and whether communication crosses nodes.



QNX services communicate through the microkernel

Microkernel

A small kernel that provides core mechanisms such as scheduling and message passing. Many services, such as drivers and filesystems, run outside the kernel as separate processes

POSIX message queues: what they are

POSIX

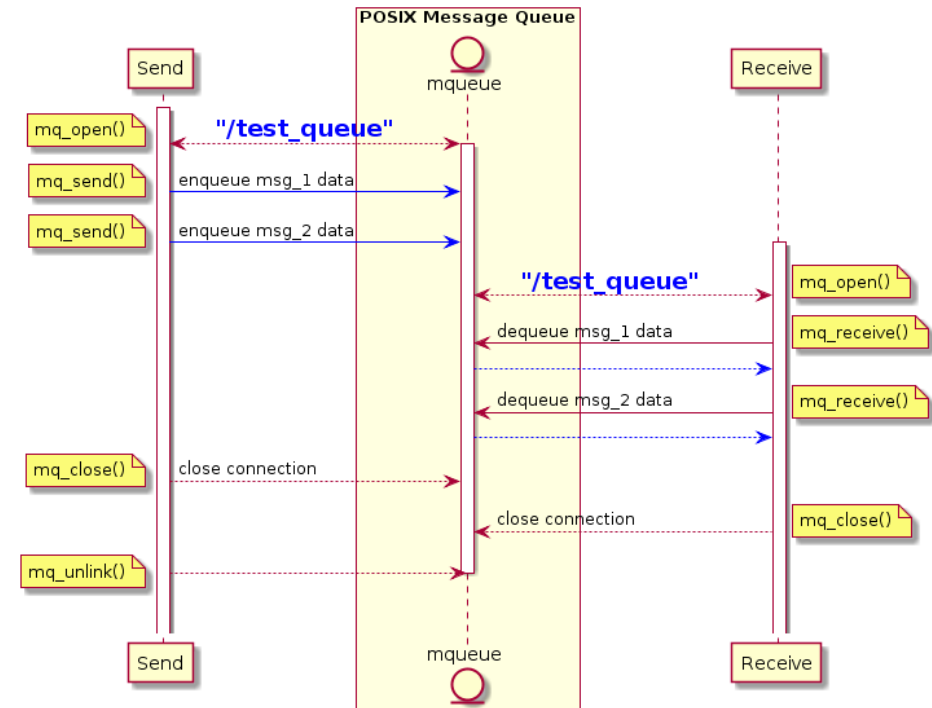
Portable Operating System Interface. It is a family of standards for operating-system APIs. POSIX mqueue code can be more portable across compliant systems

- **Store-and-forward:** the message is stored in the queue before the receiver gets it.
- By default, `mq_send()` blocks if the queue is full.
- By default, `mq_receive()` blocks if the queue is empty.
- With `O_NONBLOCK`, `mq_send()` or `mq_receive()` fails immediately instead of waiting.
- In QNX, a queue name such as `/test_queue` is represented under `/dev/mqueue`.

POSIX mqueue is **asynchronous** in the sense that the sender and receiver do not need to meet at the same time. The queue stores messages between them

Message queue

A mailbox-like IPC object. The sender puts messages into the queue. The receiver takes messages out later



POSIX message queue sequence: sender enqueues messages; receiver dequeues them later.

POSIX mqueue workflow

Function or flag	Meaning
<code>mq_open()</code>	Create or open a queue by name, such as <code>/test_queue</code> .
<code>mq_send()</code>	Copy a message into the queue. Blocks if the queue is full, unless <code>O_NONBLOCK</code> is used.
<code>mq_receive()</code>	Remove a message from the queue. Blocks if the queue is empty, unless <code>O_NONBLOCK</code> is used.
<code>mq_close()</code>	Close this process's message queue descriptor. The queue itself may still exist.
<code>mq_unlink()</code>	Remove the queue name. The queue is destroyed after all open descriptors are closed.
<code>O_CREAT</code>	Create the queue if it does not already exist.
<code>O_EXCL</code>	With <code>O_CREAT</code> , fail if the queue already exists.
<code>O_NONBLOCK</code>	Return immediately with an error instead of waiting when the queue is full or empty.

`mqd_t`

Message queue descriptor returned by `mq_open()`. Treat it like a handle for later mqueue calls.

`mq_attr`

Structure holding queue attributes, especially `mq_maxmsg` and `mq_msgsize`.

Buffer size rule

The receive buffer must be at least `mq_msgsize` bytes. If too small, `mq_receive()` fails.

Run order

If the receiver opens the queue before it exists, `mq_open()` may fail. Start the creator first, unless both sides handle creation.

Headers to remember

`<mqueue.h>` for mqueue calls, `<fcntl.h>` for `O_*` flags, and `<sys/stat.h>` for `S_IRUSR` / `S_IWUSR`.

Minimal mqqueue coding pattern

```
// Sender: create queue and enqueue messages
#include <fcntl.h>
#include <mqqueue.h>
#include <sys/stat.h>

int main(void){
    struct mq_attr attr = {0};

    attr.mq_maxmsg = 100;
    attr.mq_msgsize = 1000;

    mqd_t qd = mq_open(
        "/test_queue",
        O_RDWR | O_CREAT,
        S_IRUSR | S_IWUSR,
        &attr );

    mq_send(qd, "message 1", 10, 0);
    mq_send(qd, "done", 5, 0);

    mq_close(qd);
    mq_unlink("/test_queue");

    return 0;
}
```

```
// Receiver: open queue and dequeue messages
#include <mqqueue.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

int main(void){
    mqd_t qd = mq_open("/test_queue", O_RDONLY);

    struct mq_attr attr;
    mq_getattr(qd, &attr);

    char *buf = malloc(attr.mq_msgsize);

    while (mq_receive(qd, buf, attr.mq_msgsize, NULL) > 0) {
        printf("received: %s\n", buf);

        if (strcmp(buf, "done") == 0)
            break;
    }

    mq_close(qd);
    free(buf);

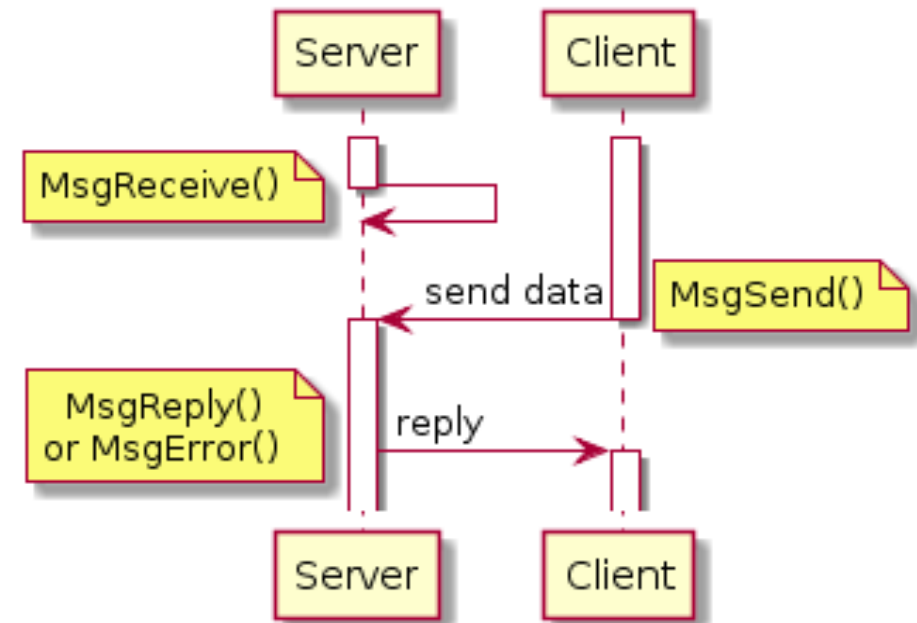
    return 0;
}
```

Important

`mq_unlink()` removes the queue name. The queue is fully destroyed only after all open descriptors are closed. Do not unlink before another process still needs to open it.

Native QNX message passing

- Native QNX message passing is synchronous IPC using a client/server model.
- Client calls `MsgSend()` and blocks until the server replies.
- Server calls `MsgReceive()` and blocks until a message or pulse arrives.
- For a normal message, the server must call `MsgReply()` or `MsgError()`; otherwise the client stays blocked.
- Blocking states to recognize in `pidin`: **SEND**, **RECEIVE** and **REPLY**.
- Messages are delivered in priority order, so design carefully to avoid priority inversion.



Send, receive and reply sequence

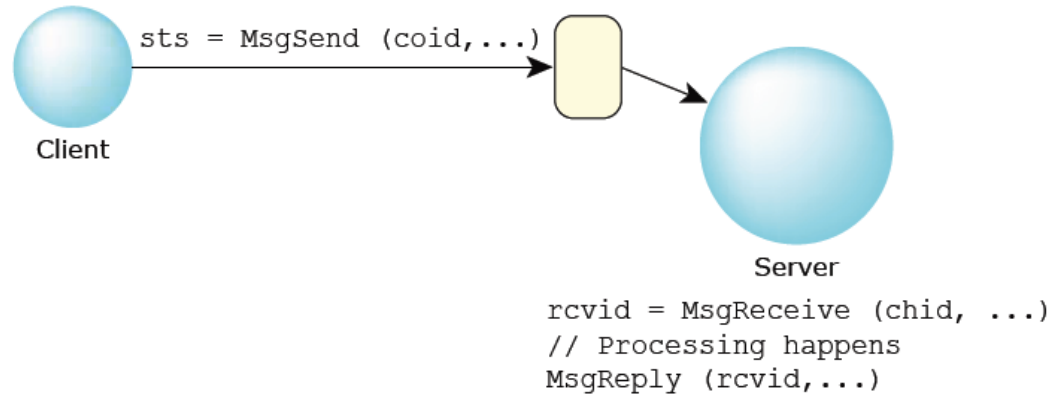
Contrast with mqueue

POSIX mqueue stores messages in a queue. Native `MsgSend()` directly synchronizes client and server through send / receive / reply.

Blocking model

Blocking is intentional: it synchronizes client and server. The client continues only after a reply, so a missing `MsgReply()` can freeze the client.

Named connections and attach points



Client/Server connection

- **Attach point:** a name registered by the server so clients can find it.
- **name_attach(NULL, ATTACH_POINT, 0):** server registers a name and creates a channel.
- **name_open(ATTACH_POINT, 0):** client opens that name and receives a connection ID.
- **Local attach** points live under `/dev/name/local`. **Global attach** points live under `/dev/name/global` when configured.
- **ATTACH_POINT** must match exactly on server and client, for example "myname".

Term	Meaning
coid	Connection ID used by the client in MsgSend().
chid	Channel ID owned by the server. MsgReceive() waits on this channel.
rcvid	Receive ID returned by MsgReceive(); pass it to MsgReply().

Server and client skeletons

```
// Server
name_attach_t *attach =
    name_attach(NULL, ATTACH_POINT, 0);

while (1) {
    int rcvid = MsgReceive(attach->chid,
                          &msg, sizeof(msg), NULL);

    if (rcvid == 0) {
        handle_pulse(msg.hdr.code);
    } else if (rcvid > 0) {
        process_message(&msg, &reply);
        MsgReply(rcvid, EOK,
                &reply, sizeof(reply));
    } else {
        break; // Error
    }
}

name_detach(attach, 0);
```

Decision rule

`rcvid > 0` means a normal message: process it and reply. `rcvid == 0` means a pulse: handle it but do not reply.

```
// Client
int coid = name_open(ATTACH_POINT, 0);

if (coid != -1) {
    MsgSend(coid,
            &msg, sizeof(msg),
            &reply, sizeof(reply));

    printf("reply: %s\n", reply.buf);

    name_close(coid);
}
```

Practical pattern

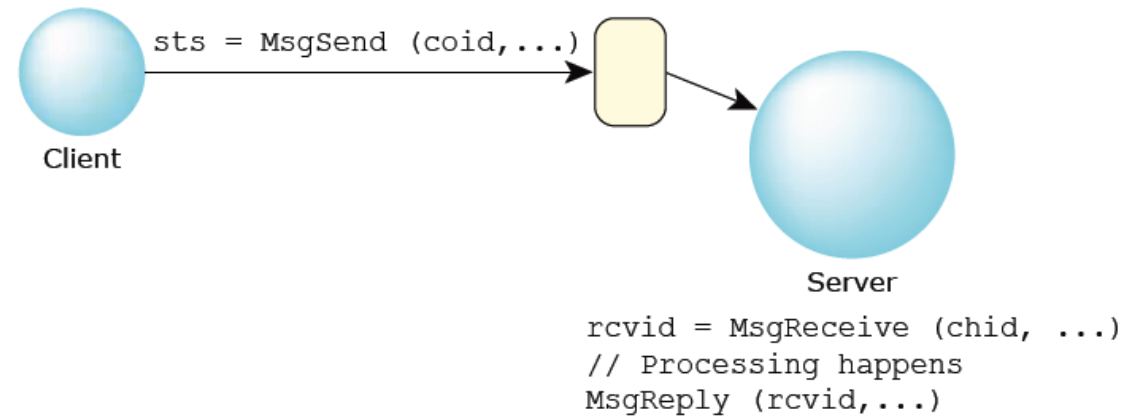
Run the server first. Then run one or more clients that connect to the same ATTACH_POINT.

Pulses

- **Pulse**: a small fixed-size QNX message used for signals or events.
- Pulses are non-blocking for the sender and do not require `MsgReply()`.
- In the server receive loop, `rcvid == 0` means a pulse was received.
- Common pulse sources include disconnect, unblock, thread death, timers and hardware events.
- `_PULSE_CODE_DISCONNECT` tells the server that a client connection closed.

Message vs pulse

Normal message: `rcvid > 0` and must be replied to. Pulse: `rcvid == 0` and must not be replied to.

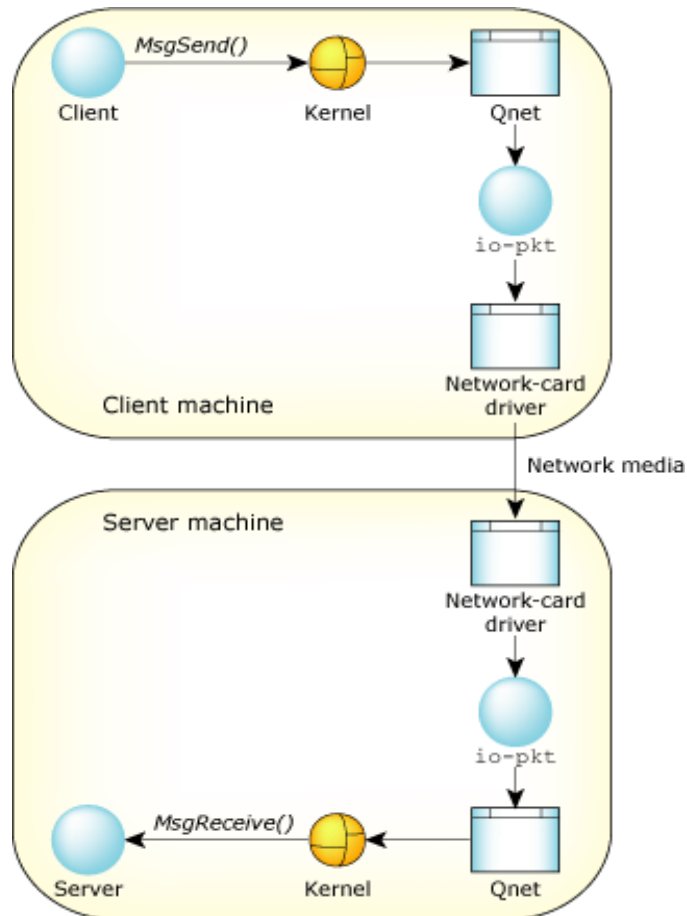


Pulse in the QNX message-passing flow

```
if (rcvid == 0) { // Pulse
    if (msg.hdr.code == _PULSE_CODE_DISCONNECT) {
        ConnectDetach(msg.hdr.scoid);
    }

    // No MsgReply() for pulses
}
```

Qnet and cross-node messaging



- Qnet lets QNX nodes see each other under /net, so remote resources can be addressed almost like local resources.
- If /net is missing on the target, load Qnet: `mount -T io-pkt lsm-qnet.so`
- A client can use a full path such as `/net/<hostname>/dev/name/local/myname`.
- Use hostname to identify the node. Each node should have a unique hostname.
- Project relevance: this is how intersection nodes and the central controller communicate.

Cross-architecture warning

An x86 VM may be 64-bit while ARM targets are 32-bit. Use fixed-width fields in message structs and never send raw pointers between nodes.

Debugging tip

Print `gethostname()`, `getpid()` and `gettid()` so you know which node and thread produced each line of output.

32-bit vs 64-bit message headers

Why this matters ?

- **Problem:** a C struct can have different size and alignment on 32-bit and 64-bit targets.
- **Example cause:** pointer-sized fields such as void* inside a header or union.
- **Result:** the receiver may decode the message incorrectly when x86 VM and ARM boards communicate.

Practical guide

- Put the header first in both request and reply structs.
- Use fixed-width integer types and explicit padding when the message crosses architectures.
- Never send raw pointers between nodes. A pointer value is only meaningful inside one process address space.

```
#include <stdint.h>

typedef struct {
    uint16_t type;
    uint16_t subtype;
    int8_t code;
    uint8_t pad[3];
    uint32_t value;
} msg_header_t;

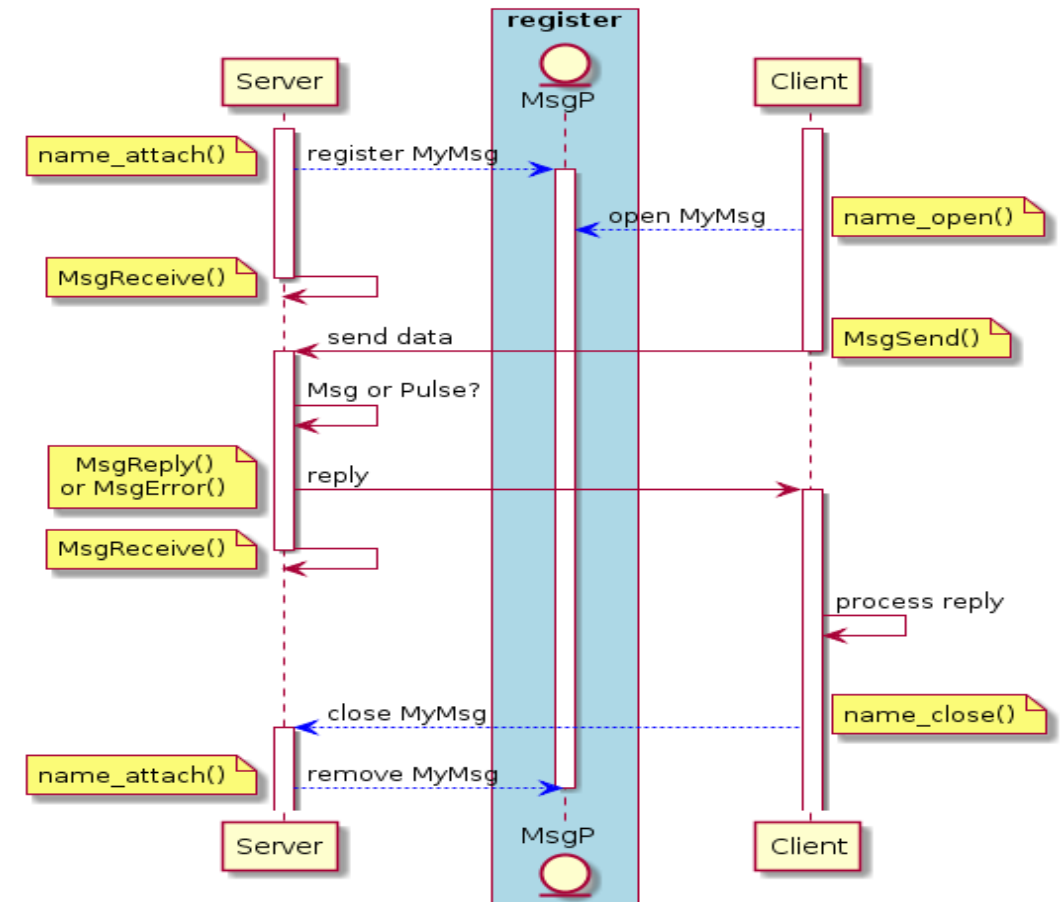
typedef struct {
    msg_header_t hdr; // must be first
    int32_t client_id;
    int32_t data;
} my_data;
```

Fixed-width message pattern

Use uint16_t, uint32_t and int32_t for fields whose size must be identical on every target.

ChannelCreate() and ConnectAttach()

- **ChannelCreate()**: server creates a channel on the same node.
- **ConnectAttach()**: client connects using the server PID and channel ID.
- This avoids name lookup, but the client must know the correct PID and CHID.
- Same message calls are used: **MsgReceive()**, **MsgSend()** and **MsgReply()**.
- Use **name_attach/name_open** when a stable named service is easier, especially across nodes.



When to use this

Good for same-node communication when the client already knows the server PID and CHID. For clarity and networking, named connections are often easier.

Three queues to understand

Receive-blocked: server waits. **Send-blocked**: clients wait for receive. **Reply-blocked**: clients wait after the server received but has not replied.

Message-passing hazards

Term	Meaning	Example
Deadlock	Threads wait forever for each other.	A sends to B while B sends to A; neither can reply.
Starvation	A thread is repeatedly denied progress.	Low priority rarely runs because high-priority work is always Ready.
Liveness	The system keeps making progress.	A waiting thread eventually gets a chance to proceed.
Priority inversion	High-priority work waits behind lower-priority work.	A high-priority client waits for a low-priority server or locked resource.
Busy-waiting	A thread loops while checking a condition.	<code>while (lock == 1) { }</code> wastes CPU.

Design rule for MsgSend()

Avoid designs where two processes synchronously send to each other at the same time. For deadlock-free message systems, keep a hierarchy: clients send upward to servers, and servers reply downward.

Synchronization Concepts

Topic	Explanation students must know
Peterson's algorithm	Mutual exclusion for exactly two processes in shared memory. Uses two intent flags and a turn variable. It uses busy-waiting.
Semaphore wait/down/P	Decreases a counter. If no resource is available, the thread blocks.
Semaphore post/up/V	Increases a counter and may wake one blocked thread.
Bounded buffer	Fixed-size buffer between producer and consumer. Producer waits when full. Consumer waits when empty.
Condition variable	Used with a mutex to sleep until a condition becomes true, such as not_empty or not_full.

```
// Condition variable pattern
pthread_mutex_lock(&m);

while (condition_is_false) {
    pthread_cond_wait(&cv, &m);
}

/* Use the shared resource here */

pthread_mutex_unlock(&m);
```

Why while, not if?

A thread can wake up even when the condition is no longer true. Recheck the condition every time after waking.

Study focus

Be ready to explain correctness, fairness, starvation, busy-waiting and bounded-buffer synchronization.

Producer-consumer bounded buffer

// Producer

```
pthread_mutex_lock(&m);

while (count == BUFFER_SIZE) {
    pthread_cond_wait(&not_full, &m);
}

insert_item(item);
count++;

pthread_cond_signal(&not_empty);
pthread_mutex_unlock(&m);
```

// Consumer

```
pthread_mutex_lock(&m);

while (count == 0) {
    pthread_cond_wait(&not_empty, &m);
}

item = remove_item();
count--;

pthread_cond_signal(&not_full);
pthread_mutex_unlock(&m);
```

Shared item / condition	Meaning
buffer, count, head, tail	Shared state protected by mutex m.
not_full	Producer waits here when the buffer is full.
not_empty	Consumer waits here when the buffer is empty.

Pattern to remember

Use one mutex to protect the shared buffer state. Use while loops for not_full and not_empty conditions.

Peterson's algorithm vs semaphore down/up

Feature	Peterson's algorithm	Semaphore wait/down and post/up
Implementation	Software only: two intent flags + a turn variable	OS primitive provided by the kernel
Number of processes	Exactly two	Any number of processes or threads
Busy-waiting	Yes: spins on flag[other] and turn	wait/down blocks; post/up wakes a waiter
Liveness	Guarantees mutual exclusion and progress for two processes	Depends on queueing and design; starvation is possible if not fair
Typical use	Teaching / strict two-process exclusion	General mutual exclusion and signalling

```
// Peterson's algorithm for process i (other = 1 - i)
flag[i] = true;           // I want to enter
turn = other;             // But let the other process go first

while (flag[other] && turn == other)
    ;                     // Busy-wait

/* Critical section */

flag[i] = false;          // Exit the critical section
```

Explanation

Peterson's algorithm is a classic algorithm for mutual exclusion between exactly two processes. Each process sets a flag to say "I want to enter", then uses the turn variable to decide who should wait if both want to enter at the same time.

Key comparison

Peterson's algorithm is useful for understanding mutual exclusion, but semaphores are more general and avoid CPU-wasting busy-waiting.

Glossary

Term	One-line meaning
RTOS	Real-Time Operating System, designed to respond within timing constraints.
QNX	A microkernel RTOS used in embedded and safety-critical systems.
POSIX	Portable Operating System Interface, a standard API family.
pthread	POSIX thread API for creating and controlling threads.
IPC	Interprocess Communication: mechanisms for data exchange and synchronization.
mqueue	POSIX message queue: mailbox-like, store-and-forward IPC.
MsgSend	QNX native call where a client sends a message and blocks for a reply.
Pulse	Small non-blocking event message received with rcvid == 0.
Attach point	Name registered by a QNX server so clients can connect.
Qnet	QNX networking layer that exposes remote nodes under /net for cross-node IPC.